Notes:

- Project proposals returned, be sure to read comments!
- HW3 due today at 5pm
- HW4 due Nov 6

Last class:

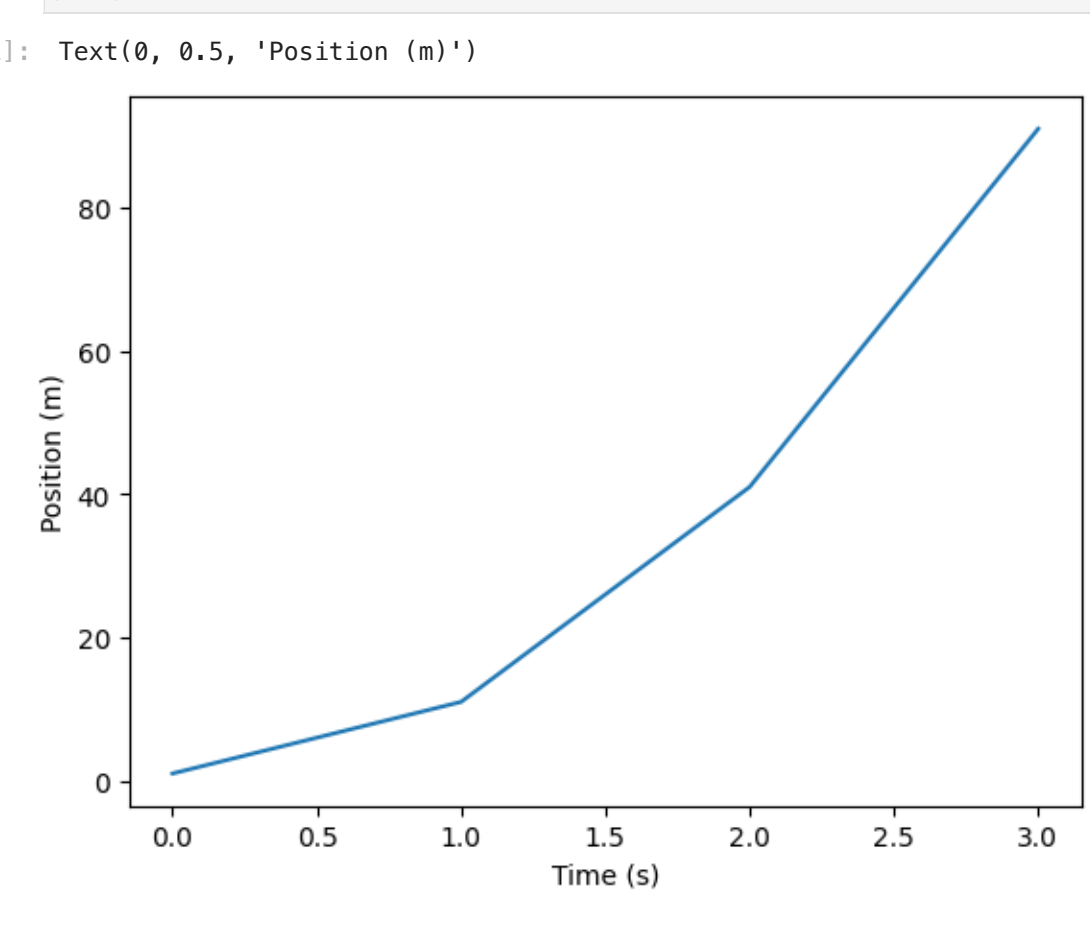
- More pandas (masking)
- lists (`append`, `extend`, `copy`)
- `for` loops and `zip`
- `while` loops
- f-strings
- started with `matplotlib`

In [1]:

```python
import matplotlib.pyplot as plt
times = [0, 1, 2, 3]
positions = [1, 11, 41, 91]
plt.plot(times, positions)
plt.xlabel("Time (s)")
plt.ylabel("Position (m)")
```

Out[1]:

```
Text(0, 0.5, 'Position (m)')
```



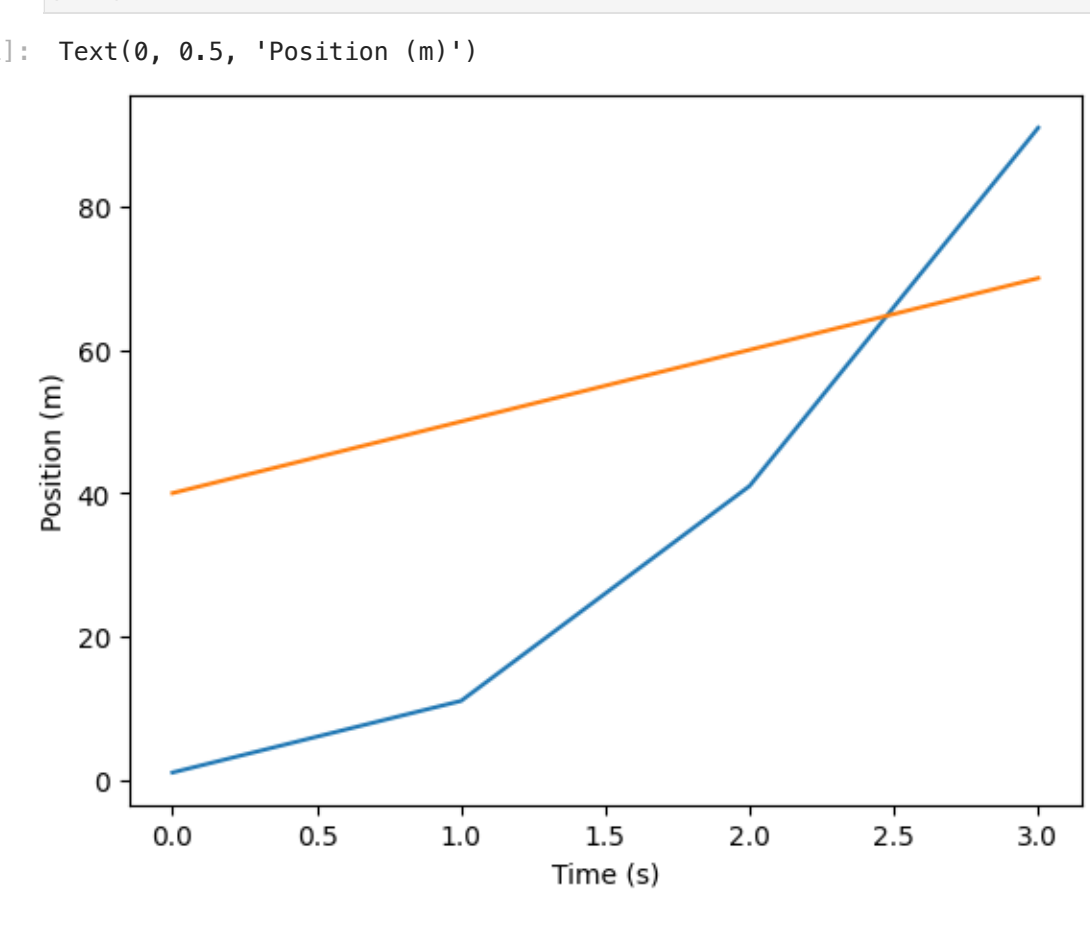
Add a second data set:

In [1]:

```python
import matplotlib.pyplot as plt
times = [0, 1, 2, 3]
positions = [1, 11, 41, 91]
positions2 = [40, 50, 60, 70]
plt.plot(times, positions)
plt.plot(times, positions2)
plt.xlabel("Time (s)")
plt.ylabel("Position (m)")
```

Out[1]:

```
Text(0, 0.5, 'Position (m)')
```



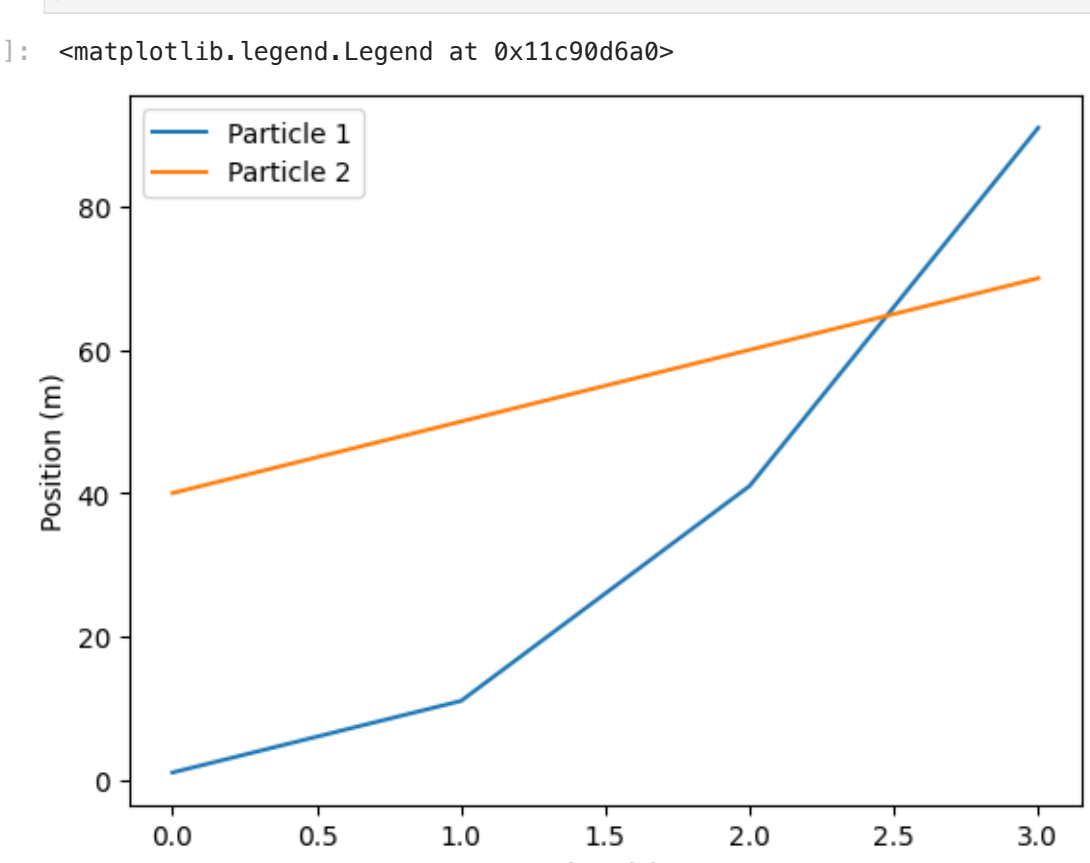
Now we need a legend, since we have more than one data line

In [2]:

```python
import matplotlib.pyplot as plt
times = [0, 1, 2, 3]
positions = [1, 11, 41, 91]
positions2 = [40, 50, 60, 70]
plt.plot(times, positions)
plt.plot(times, positions2)
plt.xlabel("Time (s)")
plt.ylabel("Position (m)")
plt.legend(['Particle 1', 'Particle 2'])
```

Out[2]:

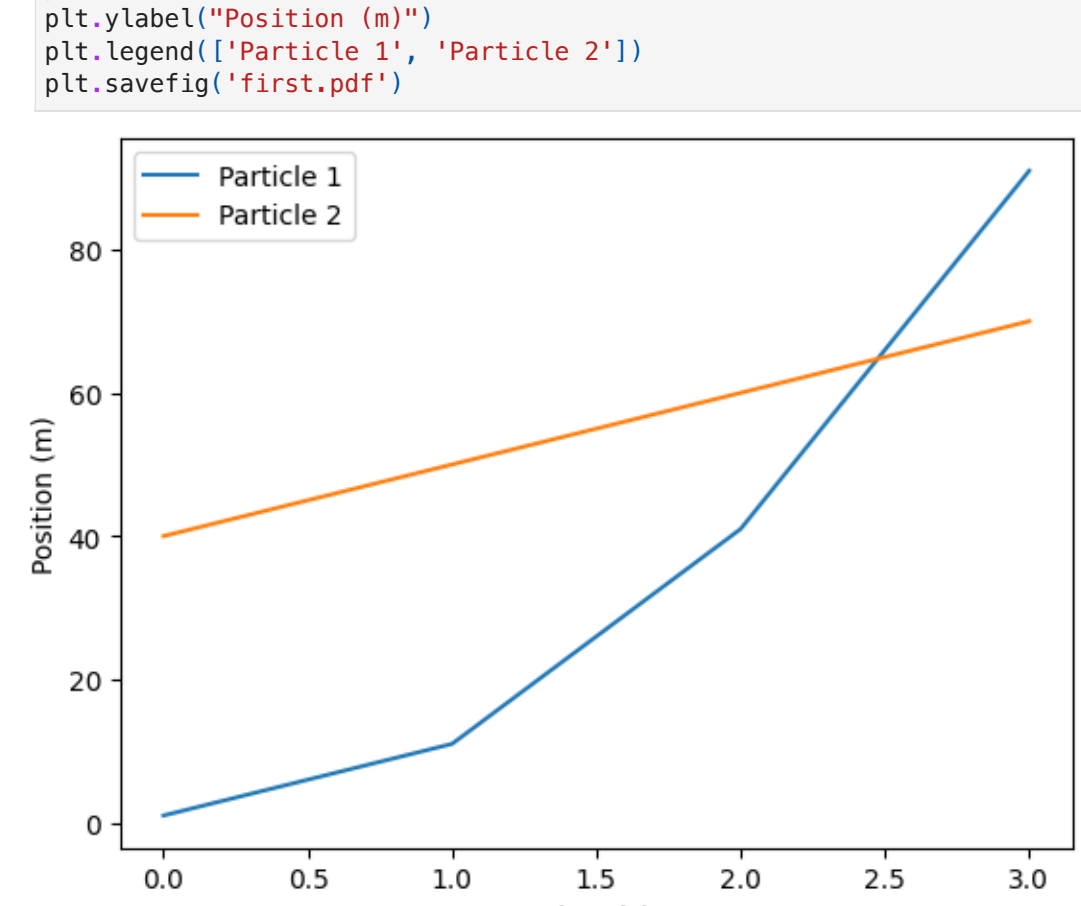
```
<matplotlib.legend.Legend at 0x11c90d6a0>
```



Now, save it to a file

In [5]:

```python
import matplotlib.pyplot as plt
times = [0, 1, 2, 3]
positions = [1, 11, 41, 91]
positions2 = [40, 50, 60, 70]
plt.plot(times, positions)
plt.plot(times, positions2)
plt.xlabel("Time (s)")
plt.ylabel("Position (m)")
plt.legend(['Particle 1', 'Particle 2'])
plt.savefig('first.pdf')
```



Can save with `plt.savefig('first.png')` or `plt.savefig('first.pdf')` . Which format should we use to save our figures?

Depends on what you want to do with it. png's are easy to use and share, but bad for use in manuscripts or reports. pdf's are a bit harder to share, but much better for use in manuscripts and reports.

Why?

Open them and zoom in... The png version shows a clear lack of resolution as we zoom in, while pdf remains completely clear.

Why is this? The png format (as well for gif and jpeg) is a "raster graphics format" based on storing discrete pixels in the image. So, as we zoom in on the image (or if we rescale it to include in a report), we *see* those pixels.

In contrast, pdf (as well as postscript or svg) are "vector graphics" formats that store images based on defining points, lines, or curves connecting those points and other geometric objects, such as circles, ellipses, polygons, text, ...

Vector graphics formats are much more functional than raster graphics formats for including in manuscripts or reports, since they can be rescaled and/or zoomed without losing quality.

Still need to be careful with plot elements, such as line widths and font sizes, but saving your (final) images as pdf is highly preferable

Note: graphs displayed in a Jupyter notebook are rendered as pngs, so saving directly form VSCode or google collab does not give good quality graphics. Use `plt.savefig('my_figure.pdf')` instead!

Let's go back to constructing plots.

Some basic things we can do with plots:

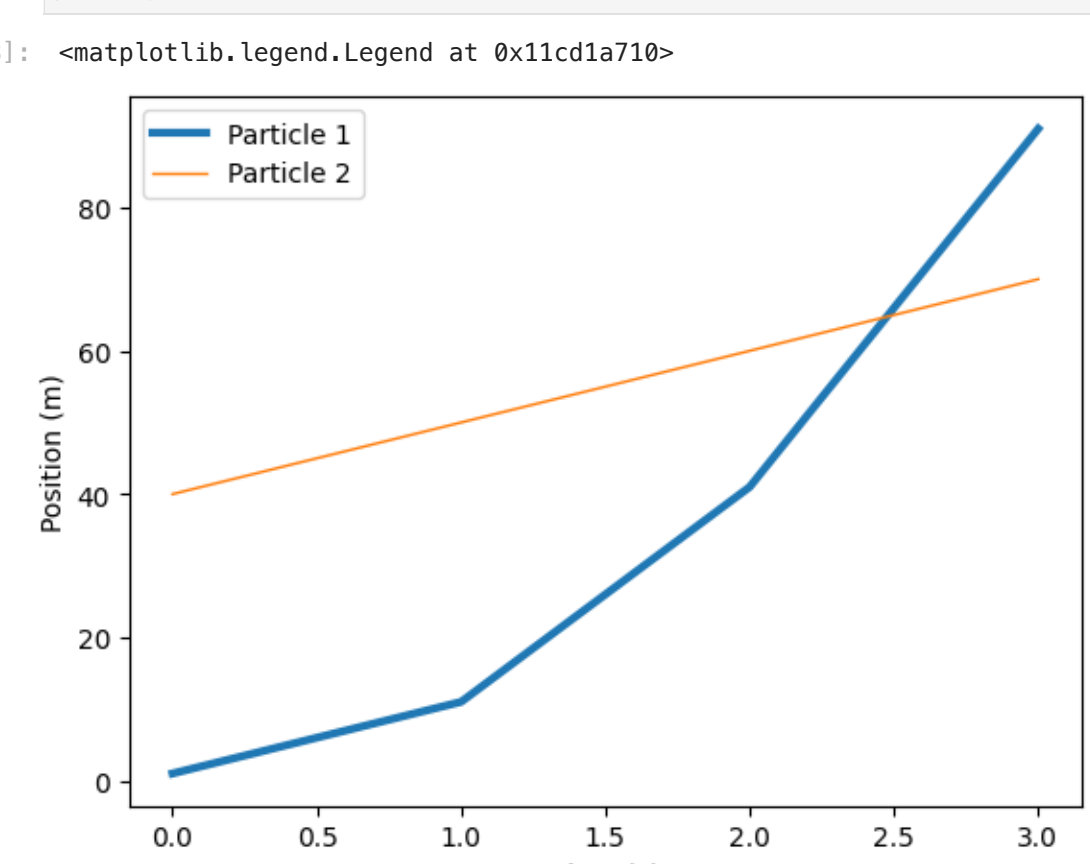
- control width of data lines
- default line width in matplotlib is 1.5, increasing this is helpful when trying to create a clearer image:

In [8]:

```python
import matplotlib.pyplot as plt
times = [0, 1, 2, 3]
positions = [1, 11, 41, 91]
positions2 = [40, 50, 60, 70]
plt.plot(times, positions, linewidth=3)
plt.plot(times, positions2, linewidth=1)
plt.xlabel("Time (s)")
plt.ylabel("Position (m)")
plt.legend(['Particle 1', 'Particle 2'])
```

Out[8]:

```
<matplotlib.legend.Legend at 0x11cd1a710>
```



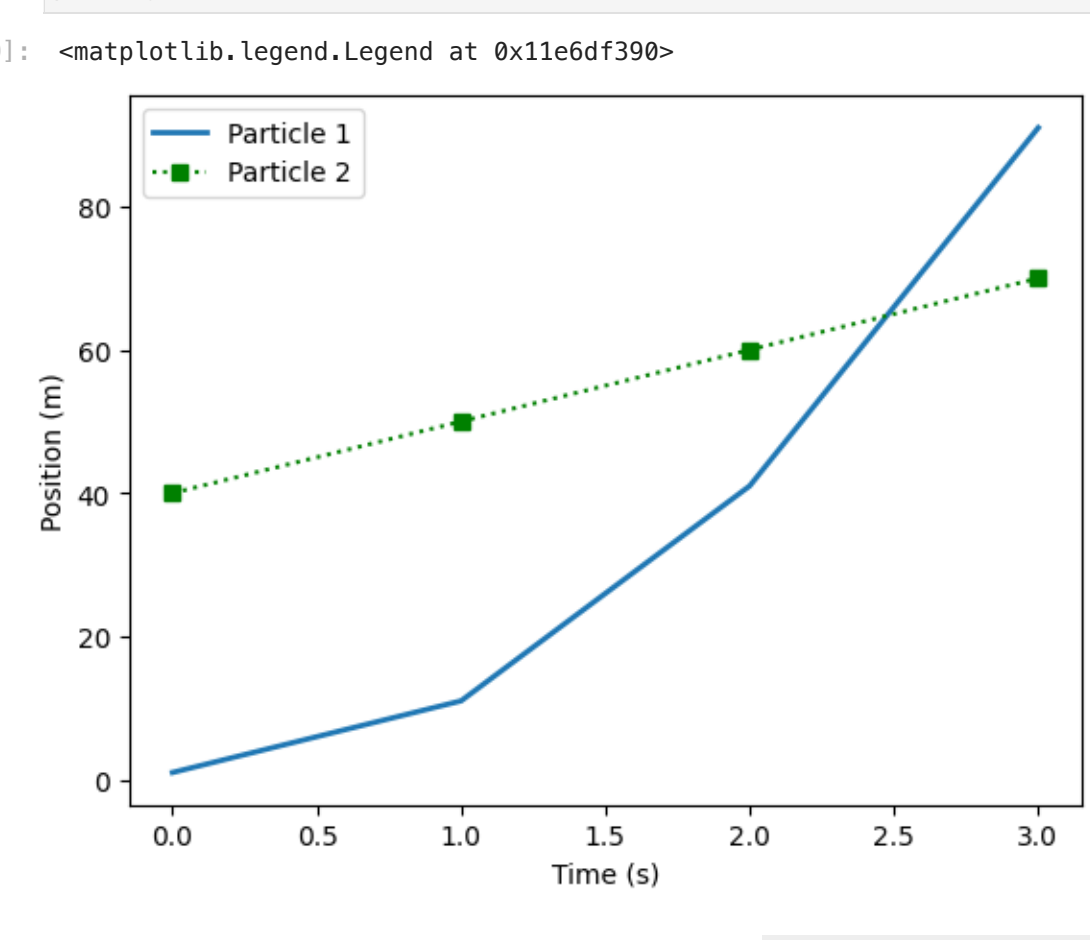
Change colors and styles of lines. Two methods to do this. First of using a format string that follows the data:

In [9]:

```python
import matplotlib.pyplot as plt
times = [0, 1, 2, 3]
positions = [1, 11, 41, 91]
positions2 = [40, 50, 60, 70]
plt.plot(times, positions, linewidth=2)
plt.plot(times, positions2, 's:g')
plt.xlabel("Time (s)")
plt.ylabel("Position (m)")
plt.legend(['Particle 1', 'Particle 2'])
```

Out[9]:

```
<matplotlib.legend.Legend at 0x11e6df390>
```



Format strings are convenient, but cryptic. Take the form `[marker][line][color]` , where

- `marker` is a character denoting what sort of marks to place at the data points. In the above example, use `s` for squares, could also use `o` for circles, `*` for stars, `x` for x's, etc.
- `line` is a character denoting line style, with `-` for solid, `--` for dashed, `-.` is for dash-dotted, `:` is for dotted line (as above), and ` ` for no line
- `color` is a character denoting the colour for the line/markers, such as `b` for blue, `y` for yellow, `k` for black, etc.

All of these format elements are optional and, if omitted, default to

- no markers
- solid lines
- colours that rotate through blue, orange, green, red, purple, ...

Way more control if you use keyword arguments (kwargs) to set these values:

- for markers, have `marker` that takes a format character as above, but also
    - `markersize` to control size (default is 6)
    - `markeredgecolor` to set colour for outside of marker (defaults to same as line)
    - `markeredgewidth` to set width of edge (default is 1)
    - `markerfacecolor` to set color for inside of the marker (defaults to same as line)
    - `markevery` to set which points are marked (default is all points). Set it to number `N` to mark ever Nth point from first or pair `(i,N)` to mark every Nth point from ith point. Other options include marking specific data points by index, or providing True/False array indicating which data points to mark.

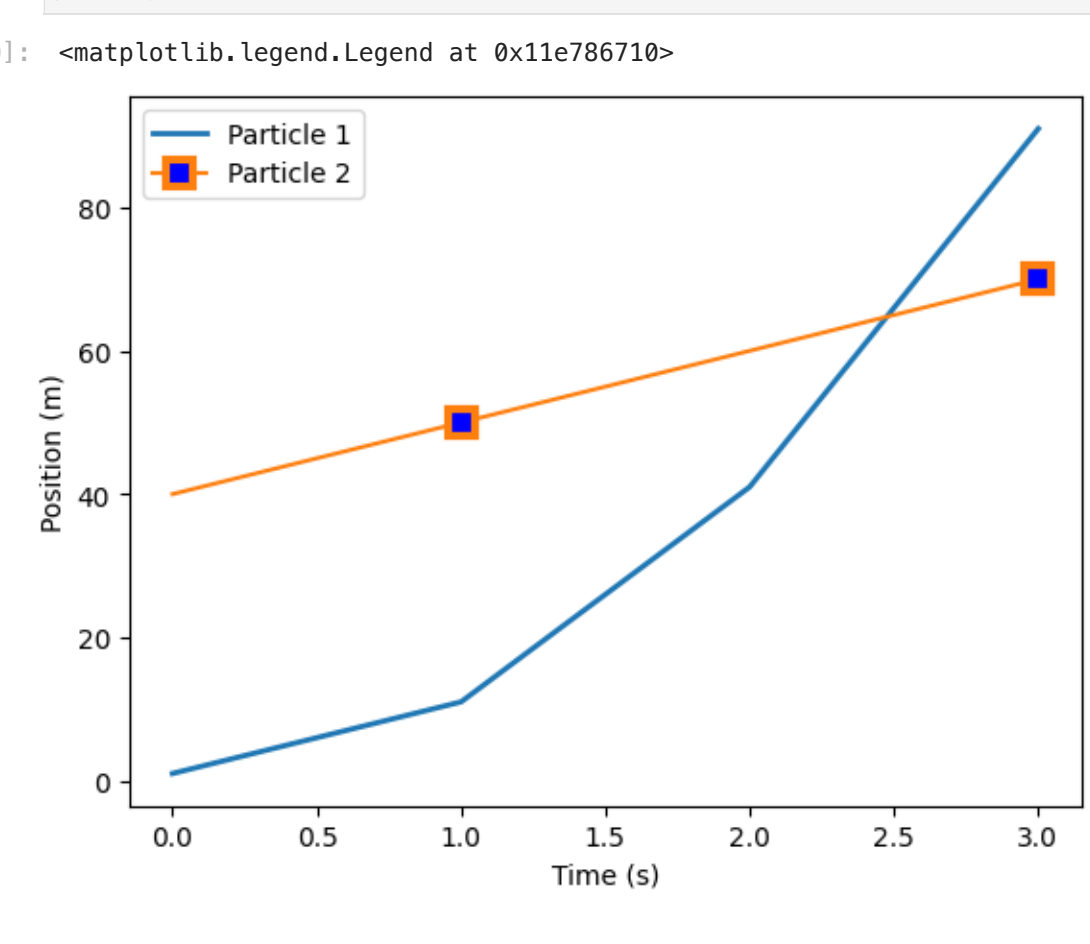
This lets us mark data points like:

In [10]:

```python
plt.plot(times, positions, linewidth=2)
plt.plot(times, positions2, marker = 's', markersize = 10, markerfacecolor = 'b', markeredgewidth = 3, markevery = (1,2))
plt.xlabel("Time (s)")
plt.ylabel("Position (m)")
plt.legend(['Particle 1', 'Particle 2'])
```

Out[10]:

```
<matplotlib.legend.Legend at 0x11e786710>
```



- `linestyle` is generally the same as in the format string, with added ability to customize dot-dash patterns by providing lengths for each dash and space
- `color` allows much more control of colours using either RGB tuples, X11 colour names (e.g., `cornflowerblue`) or names from xkcd color survey (e.g., `xkcd:pumpkin`)

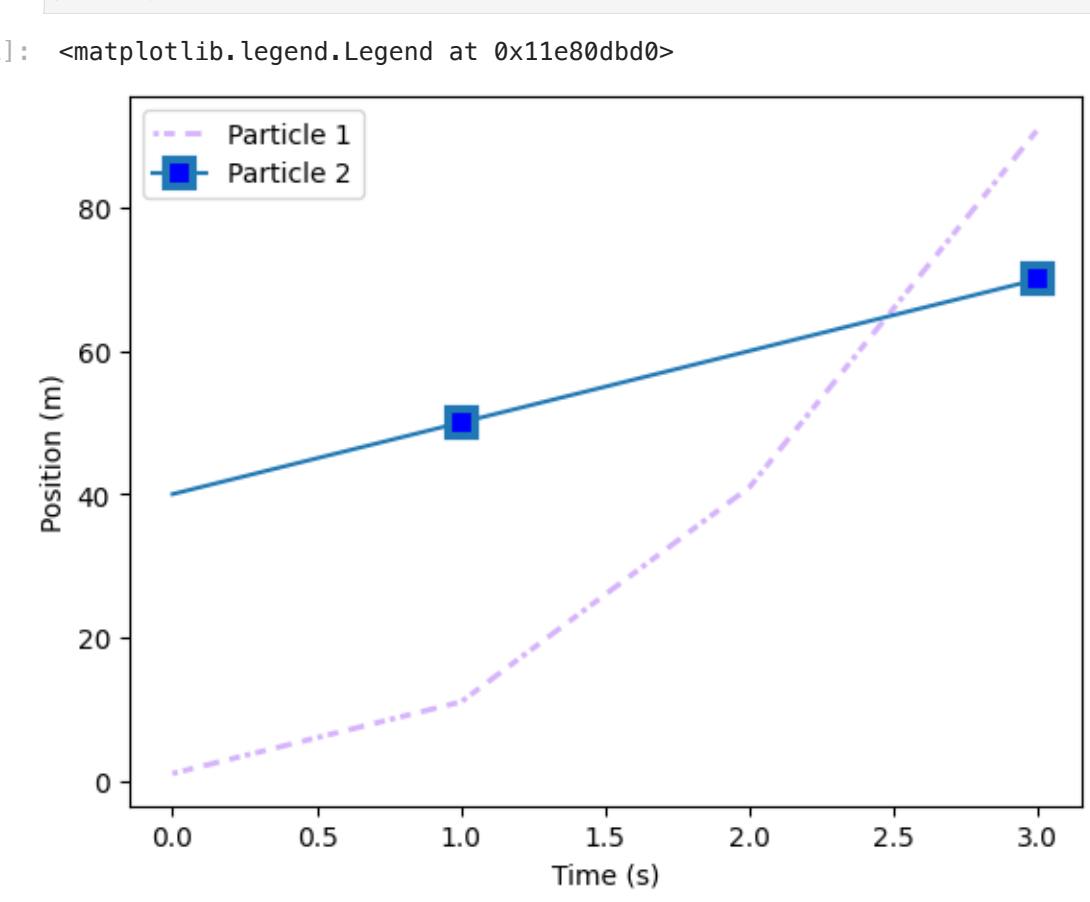
This allows us to add these to our plots:

In [11]:

```python
plt.plot(times, positions, linewidth=2, linestyle=(0,(1,1,2,2,3,2,2,1)), color='xkcd:light violet')
plt.plot(times, positions2, marker = 's', markersize = 10, markerfacecolor = 'b', markeredgewidth = 3, markevery = (1,2))
plt.xlabel("Time (s)")
plt.ylabel("Position (m)")
plt.legend(['Particle 1', 'Particle 2'])
```

Out[11]:

```
<matplotlib.legend.Legend at 0x11e80dbd0>
```



Notice that if you only specify colours for some lines, others cycle through default colour cycle in fixed order. So first line without a specified colour is blue, next one is orange, etc.

Can also directly label our data in plot command for legends. Add `label = 'my label'` to each line in plot command, then remove the list of labels from `plt.legend`

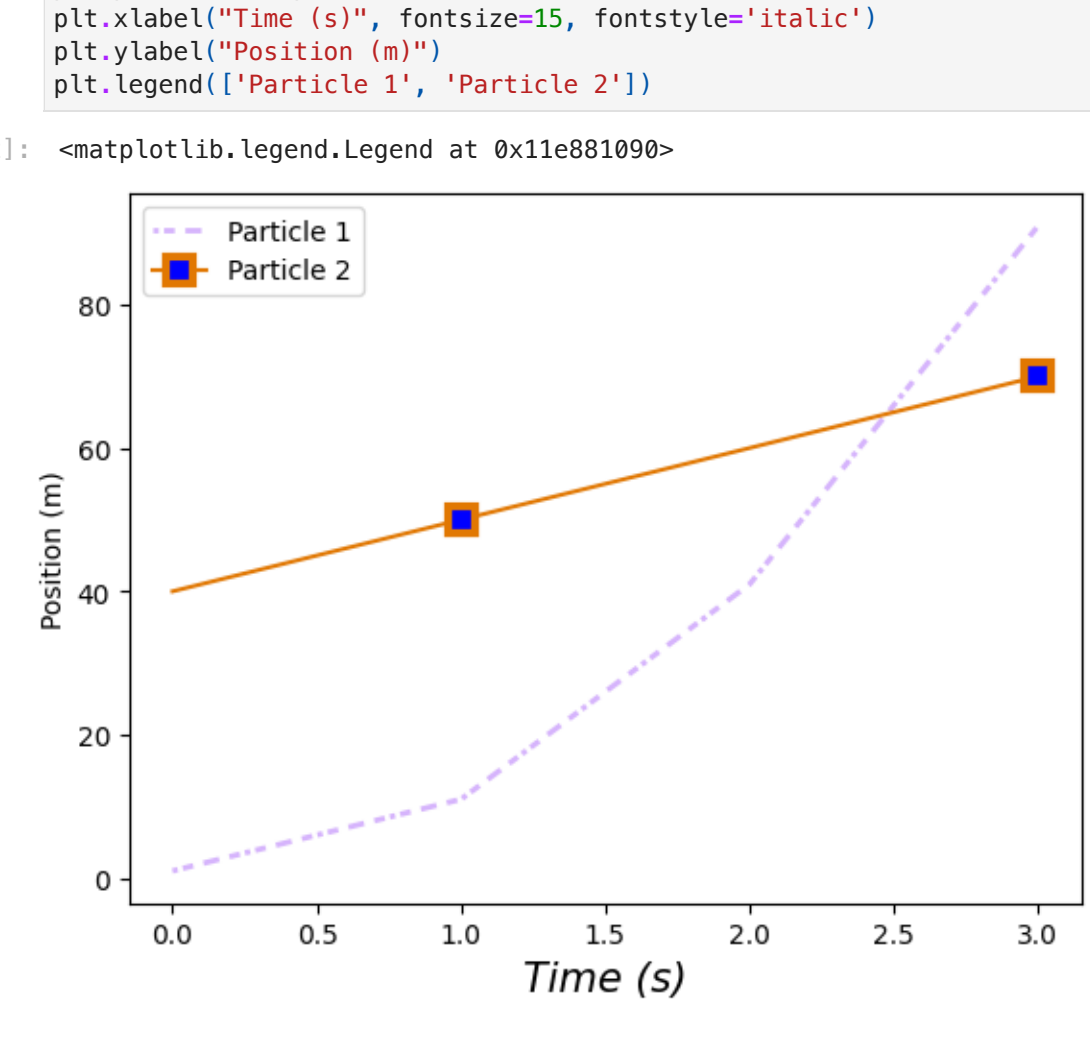
Can also adjust other plot aspects, such as increasing font sizes (or changing the font) for axis labels, legend, etc...

In [12]:

```python
plt.plot(times, positions, linewidth=2, linestyle=(0,(1,1,2,2,3,2,2,1)), color='xkcd:light violet')
plt.plot(times, positions2, marker = 's', markersize = 10, markerfacecolor = 'b', markeredgewidth = 3, markevery = (1,2), color='xkcd:pumpkin')
plt.xlabel("Time (s)", fontsize=15, fontstyle='italic')
plt.ylabel("Position (m)")
plt.legend(['Particle 1', 'Particle 2'])
```

Out[12]:

```
<matplotlib.legend.Legend at 0x11e881090>
```



With all of these options comes the issue of consistency. How do we ensure that **all** text elements use the same font, size, style, etc. How do we make consistent lines across figures, and so on.

matplotlib stores its runtime configuration (`rc`) in a Python *dictionary*, so to update configuration for all plots, just need to update values in this dictionary.

What's a dictionary?

A dictionary is a set of pairs of keys and values. Each key must be unique in the dictionary and immutable (so strings, numbers, or tuples are okay, but lists are not). Values, however, can be anything.

A few ways to create dictionaries:

In [13]:

```python
my_dict = {}
my_dict['dogs'] = 3
my_dict[2] = 'cats'
my_dict['elephant'] = [3,2]
print(my_dict)
```

```
{'dogs': 3, 2: 'cats', 'elephant': [3, 2]}
```

This starts from an empty dictionary, then adds items by specifying each key (here `'dogs'`, `2`, and `'elephant'` and its value). Can also initialize values on same line as we declare a dictionary:

In [14]:

```python
my_dict2 = {'dogs':2,
            2: 'cats',
            'elephants': [3,2]}
print(my_dict2)
```

```
{'dogs': 2, 2: 'cats', 'elephants': [3, 2]}
```

Can also create a dictionary by taking list of key-value pairs

In [16]:

```python
my_dict3 = dict([('dogs',2),(2,'cats'),('elephants', [3,2])])
print(my_dict3)
```

```
{'dogs': 2, 2: 'cats', 'elephants': [3, 2]}
```

Can check if a key appears in a dictionary with `in` :

In [17]:

```python
print('dogs' in my_dict)
```

```
True
```

In [18]:

```python
print('cats' in my_dict)
```

```
False
```

List keys or values with

In [19]:

```python
print(my_dict.keys())
print(my_dict.values())
```

```
dict_keys(['dogs', 2, 'elephant'])
dict_values([3, 'cats', [3, 2]])
```

Get data:

In [20]:

```python
print(my_dict.get('elephant'))
```

```
[3, 2]
```

Get a list of (key,value) pairs:

In [21]:

```python
print(my_dict.items())
```

```
dict_items([('dogs', 3), (2, 'cats'), ('elephant', [3, 2])])
```

These give us ways of looping over entries in dictionary:

In [22]:

```python
for key, value in my_dict.items():
    print(f"Key {key} has value {value}")
```

```
Key dogs has value 3
Key 2 has value cats
Key elephant has value [3, 2]
```

Overwrite a value just with assignment

In [23]:

```python
my_dict['dogs'] = 'woof'
print(my_dict.items())
```

```
dict_items([('dogs', 'woof'), (2, 'cats'), ('elephant', [3, 2])])
```

See methods on dictionaries with

In [24]:

```python
help(dict)
```

```
Help on class dict in module builtins:

class dict(object)
 |  dict() -> new empty dictionary
 |  dict(mapping) -> new dictionary initialized from a mapping object's
 |      (key, value) pairs
 |  dict(iterable) -> new dictionary initialized as if via:
 |      d = {}
 |      for k, v in iterable:
 |          d[k] = v
 |  dict(**kwargs) -> new dictionary initialized with the name=value pairs
 |      in the keyword argument list.  For example:  dict(one=1, two=2)
 |
 |  Methods defined here:
 |
 |  __contains__(self, key, /)
 |      True if the dictionary has the specified key, else False.
 |
 |  __delitem__(self, key, /)
 |      Delete self[key].
 |
 |  __eq__(self, value, /)
 |      Return self==value.
 |
 |  __ge__(self, value, /)
 |      Return self>=value.
 |
 |  __getattribute__(self, name, /)
 |      Return getattr(self, name).
 |
 |  __getitem__(self, key, /)
 |      Return self[key].
 |
 |  __gt__(self, value, /)
 |      Return self>value.
 |
 |  __init__(self, /, *args, **kwargs)
 |      Initialize self.  See help(type(self)) for accurate signature.
 |
 |  __ior__(self, value, /)
 |      Return self|=value.
 |
 |  __iter__(self, /)
 |      Implement iter(self).
 |
 |  __le__(self, value, /)
 |      Return self<=value.
 |
 |  __len__(self, /)
 |      Return len(self).
 |
 |  __lt__(self, value, /)
 |      Return self<value.
 |
 |  __ne__(self, value, /)
 |      Return self!=value.
 |
 |  __or__(self, value, /)
 |      Return self|value.
 |
 |  __repr__(self, /)
 |      Return repr(self).
 |
 |  __reversed__(self, /)
 |      Return a reverse iterator over the dict keys.
 |
 |  __ror__(self, value, /)
 |      Return value|self.
 |
 |  __setitem__(self, key, value, /)
 |      Set self[key] to value.
 |
 |  __sizeof__(self, /)
 |      Return the size of the dict in memory, in bytes.
 |
 |  clear(self, /)
 |      Remove all items from the dict.
 |
 |  copy(self, /)
 |      Return a shallow copy of the dict.
 |
 |  get(self, key, default=None, /)
 |      Return the value for key if key is in the dictionary, else default.
 |
 |  items(self, /)
 |      Return a set-like object providing a view on the dict's items.
 |
 |  keys(self, /)
 |      Return a set-like object providing a view on the dict's keys.
 |
 |  pop(self, key, default=<unrepresentable>, /)
 |      D.pop(k[,d]) -> v, remove specified key and return the corresponding value.
 |
 |      If the key is not found, return the default if given; otherwise,
 |      raise a KeyError.
 |
 |  popitem(self, /)
 |      Remove and return a (key, value) pair as a 2-tuple.
 |
 |      Pairs are returned in LIFO (last-in, first-out) order.
 |      Raises KeyError if the dict is empty.
 |
 |  setdefault(self, key, default=None, /)
 |      Insert key with a value of default if key is not in the dictionary.
 |
 |      Return the value for key if key is in the dictionary, else default.
 |
 |  update(...)
 |      D.update([E, ]**F) -> None.  Update D from mapping/iterable E and F.
 |      If E is present and has a .keys() method, then does:  for k in E.keys(): D[k] = E[k]
 |      If E is present and lacks a .keys() method, then does:  for k, v in E: D[k] = v
 |      In either case, this is followed by: for k in F:  D[k] = F[k]
 |
 |  values(self, /)
 |      Return an object providing a view on the dict's values.
 |
 |  ----------------------------------------------------------------------
 |  Class methods defined here:
 |
 |  __class_getitem__(object, /)
 |      See PEP 585
 |
 |  fromkeys(iterable, value=None, /)
 |      Create a new dictionary with keys from iterable and values set to value.
 |
 |  ----------------------------------------------------------------------
 |  Static methods defined here:
 |
 |  __new__(*args, **kwargs)
 |      Create and return a new object.  See help(type) for accurate signature.
 |
 |  ----------------------------------------------------------------------
 |  Data and other attributes defined here:
 |
 |  __hash__ = None
```

Back to matplotlib defaults

In [25]:

```python
import matplotlib as mpl
for key, value in mpl.rcParams.items():
    print(f"Key {key} has value {value}")
```

```
Key _internal.classic_mode has value False
Key agg.path.chunksize has value 0
Key animation.bitrate has value -1
Key animation.codec has value h264
Key animation.convert_args has value ['-layers', 'OptimizePlus']
Key animation.convert_path has value convert
Key animation.embed_limit has value 20.0
Key animation.ffmpeg_args has value []
Key animation.ffmpeg_path has value ffmpeg
Key animation.frame_format has value png
Key animation.html has value none
Key animation.writer has value ffmpeg
Key axes.autolimit_mode has value data
Key axes.axisbelow has value line
Key axes.edgecolor has value black
Key axes.facecolor has value white
Key axes.formatter.limits has value [-5, 6]
Key axes.formatter.min_exponent has value 0
Key axes.formatter.offset_threshold has value 4
Key axes.formatter.use_locale has value False
Key axes.formatter.use_mathtext has value False
Key axes.formatter.useoffset has value True
Key axes.grid has value False
Key axes.grid.axis has value both
Key axes.grid.which has value major
Key axes.labelcolor has value black
Key axes.labelpad has value 4.0
Key axes.labelsize has value medium
Key axes.labelweight has value normal
Key axes.linewidth has value 0.8
Key axes.prop_cycle has value cycler('color', ['#1f77b4', '#ff7f0e', '#2ca02c', '#d62728', '#9467bd', '#8c564b', '#e377c2', '#7f7f7f', '#bcbd22', '#17becf'])
Key axes.spines.bottom has value True
Key axes.spines.left has value True
Key axes.spines.right has value True
Key axes.spines.top has value True
Key axes.titlecolor has value auto
Key axes.titlelocation has value center
Key axes.titlepad has value 6.0
Key axes.titlesize has value large
Key axes.titleweight has value normal
Key axes.titley has value None
Key axes.unicode_minus has value True
Key axes.xmargin has value 0.05
Key axes.ymargin has value 0.05
Key axes.zmargin has value 0.05
Key axes3d.automargin has value False
Key axes3d.grid has value True
Key axes3d.mouserotationstyle has value arcball
Key axes3d.trackballborder has value 0.2
Key axes3d.trackballsize has value 0.667
Key axes3d.xaxis.panecolor has value (0.95, 0.95, 0.95, 0.5)
Key axes3d.yaxis.panecolor has value (0.9, 0.9, 0.9, 0.5)
Key axes3d.zaxis.panecolor has value (0.925, 0.925, 0.925, 0.5)
Key backend has value module://matplotlib_inline.backend_inline
Key backend_fallback has value True
Key boxplot.bootstrap has value None
Key boxplot.boxprops.color has value black
Key boxplot.boxprops.linestyle has value -
Key boxplot.boxprops.linewidth has value 1.0
Key boxplot.capprops.color has value black
Key boxplot.capprops.linestyle has value -
Key boxplot.capprops.linewidth has value 1.0
Key boxplot.flierprops.color has value black
Key boxplot.flierprops.linestyle has value none
Key boxplot.flierprops.linewidth has value 1.0
Key boxplot.flierprops.marker has value o
Key boxplot.flierprops.markeredgecolor has value black
Key boxplot.flierprops.markeredgewidth has value 1.0
Key boxplot.flierprops.markerfacecolor has value none
Key boxplot.flierprops.markersize has value 6.0
Key boxplot.meanline has value False
Key boxplot.meanprops.color has value C2
Key boxplot.meanprops.linestyle has value --
Key boxplot.meanprops.linewidth has value 1.0
Key boxplot.meanprops.marker has value ^
Key boxplot.meanprops.markeredgecolor has value C2
Key boxplot.meanprops.markerfacecolor has value C2
Key boxplot.meanprops.markersize has value 6.0
Key boxplot.medianprops.color has value C1
Key boxplot.medianprops.linestyle has value -
Key boxplot.medianprops.linewidth has value 1.0
Key boxplot.notch has value False
Key boxplot.patchartist has value False
Key boxplot.showbox has value True
Key boxplot.showcaps has value True
Key boxplot.showfliers has value True
Key boxplot.showmeans has value False
Key boxplot.vertical has value True
Key boxplot.whiskerprops.color has value black
Key boxplot.whiskerprops.linestyle has value -
Key boxplot.whiskerprops.linewidth has value 1.0
Key boxplot.whiskers has value 1.5
Key contour.algorithm has value mpl2014
Key contour.corner_mask has value True
Key contour.linewidth has value None
Key contour.negative_linestyle has value dashed
Key date.autoformatter.day has value %Y-%m-%d
Key date.autoformatter.hour has value %m-%d %H
Key date.autoformatter.microsecond has value %M:%S.%f
Key date.autoformatter.minute has value %d %H:%M
Key date.autoformatter.month has value %Y-%m
Key date.autoformatter.second has value %H:%M:%S
Key date.autoformatter.year has value %Y
Key date.converter has value auto
Key date.epoch has value 1970-01-01T00:00:00
Key date.interval_multiples has value True
Key docstring.hardcopy has value False
Key errorbar.capsize has value 0.0
Key figure.autolayout has value False
Key figure.constrained_layout.h_pad has value 0.04167
Key figure.constrained_layout.hspace has value 0.02
Key figure.constrained_layout.use has value False
Key figure.constrained_layout.w_pad has value 0.04167
Key figure.constrained_layout.wspace has value 0.02
Key figure.dpi has value 100.0
Key figure.edgecolor has value white
Key figure.facecolor has value white
Key figure.figsize has value [6.4, 4.8]
Key figure.frameon has value True
Key figure.hooks has value []
Key figure.labelsize has value large
Key figure.labelweight has value normal
Key figure.max_open_warning has value 20
Key figure.raise_window has value True
Key figure.subplot.bottom has value 0.11
Key figure.subplot.hspace has value 0.2
Key figure.subplot.left has value 0.125
Key figure.subplot.right has value 0.9
Key figure.subplot.top has value 0.88
Key figure.subplot.wspace has value 0.2
Key figure.titlesize has value large
Key figure.titleweight has value normal
Key font.cursive has value ['Apple Chancery', 'Textile', 'Zapf Chancery', 'Sand', 'Script MT', 'Felipa', 'Comic Neue', 'Comic Sans MS', 'cursive']
Key font.family has value ['sans-serif']
Key font.fantasy has value ['Chicago', 'Charcoal', 'Impact', 'Western', 'xkcd script', 'fantasy']
Key font.monospace has value ['DejaVu Sans Mono', 'Bitstream Vera Sans Mono', 'Computer Modern Typewriter', 'Andale Mono', 'Nimbus Mono L', 'Courier New', 'Courier', 'Fixed', '
Terminal', 'monospace']
Key font.sans-serif has value ['DejaVu Sans', 'Bitstream Vera Sans', 'Computer Modern Sans Serif', 'Lucida Grande', 'Verdana', 'Geneva', 'Lucid', 'Arial', 'Helvetica', 'Avant G
arde', 'sans-serif']
Key font.serif has value ['DejaVu Serif', 'Bitstream Vera Serif', 'Computer Modern Roman', 'New Century Schoolbook', 'Century Schoolbook L', 'Utopia', 'ITC Bookman', 'Bookman',
'Nimbus Roman No9 L', 'Times New Roman', 'Times', 'Palatino', 'Charter', 'serif']
Key font.size has value 10.0
Key font.stretch has value normal
Key font.style has value normal
Key font.variant has value normal
Key font.weight has value normal
Key grid.alpha has value 1.0
Key grid.color has value #b0b0b0
Key grid.linestyle has value -
Key grid.linewidth has value 0.8
Key hatch.color has value black
Key hatch.linewidth has value 1.0
Key hist.bins has value 10
Key image.aspect has value equal
Key image.cmap has value viridis
Key image.composite_image has value True
Key image.interpolation has value auto
Key image.interpolation_stage has value auto
Key image.lut has value 256
Key image.origin has value upper
Key image.resample has value True
Key interactive has value True
Key keymap.back has value ['left', 'c', 'backspace', 'MouseButton.BACK']
Key keymap.copy has value ['ctrl+c', 'cmd+c']
Key keymap.forward has value ['right', 'v', 'MouseButton.FORWARD']
Key keymap.fullscreen has value ['f', 'ctrl+f']
Key keymap.grid has value ['g']
Key keymap.grid_minor has value ['G']
Key keymap.help has value ['f1']
Key keymap.home has value ['h', 'r', 'home']
Key keymap.pan has value ['p']
Key keymap.quit has value ['ctrl+w', 'cmd+w', 'q']
Key keymap.quit_all has value []
Key keymap.save has value ['s', 'ctrl+s']
Key keymap.xscale has value ['k', 'L']
Key keymap.yscale has value ['l']
Key keymap.zoom has value ['o']
Key legend.borderaxespad has value 0.5
Key legend.borderpad has value 0.4
Key legend.columnspacing has value 2.0
Key legend.edgecolor has value 0.8
Key legend.facecolor has value inherit
Key legend.fancybox has value True
Key legend.fontsize has value medium
Key legend.framealpha has value 0.8
Key legend.frameon has value True
Key legend.handleheight has value 0.7
Key legend.handlelength has value 2.0
Key legend.handletextpad has value 0.8
Key legend.labelcolor has value None
Key legend.labelspacing has value 0.5
Key legend.loc has value best
Key legend.markerscale has value 1.0
Key legend.numpoints has value 1
Key legend.scatterpoints has value 1
Key legend.shadow has value False
Key legend.title_fontsize has value None
Key lines.antialiased has value True
Key lines.color has value C0
Key lines.dash_capstyle has value CapStyle.butt
Key lines.dash_joinstyle has value JoinStyle.round
Key lines.dashdot_pattern has value [6.4, 1.6, 1.0, 1.6]
Key lines.dashed_pattern has value [3.7, 1.6]
Key lines.dotted_pattern has value [1.0, 1.65]
Key lines.linestyle has value -
Key lines.linewidth has value 1.5
Key lines.marker has value None
Key lines.markeredgecolor has value auto
Key lines.markeredgewidth has value 1.0
Key lines.markerfacecolor has value auto
Key lines.markersize has value 6.0
Key lines.scale_dashes has value True
Key lines.solid_capstyle has value CapStyle.projecting
Key lines.solid_joinstyle has value JoinStyle.round
Key macosx.window_mode has value system
Key markers.fillstyle has value full
Key mathtext.bf has value sans:bold
Key mathtext.bfit has value sans:italic:bold
Key mathtext.cal has value cursive
Key mathtext.default has value it
Key mathtext.fallback has value cm
Key mathtext.fontset has value dejavusans
Key mathtext.it has value sans:italic
Key mathtext.rm has value sans
Key mathtext.sf has value sans
Key mathtext.tt has value monospace
Key patch.antialiased has value True
Key patch.edgecolor has value black
Key patch.facecolor has value C0
Key patch.force_edgecolor has value False
Key patch.linewidth has value 1.0
Key path.effects has value []
Key path.simplify has value True
Key path.simplify_threshold has value 0.111111111111
Key path.sketch has value None
Key path.snap has value True
Key pcolor.shading has value auto
Key pcolormesh.snap has value True
Key pdf.compression has value 6
Key pdf.fonttype has value 3
Key pdf.inheritcolor has value False
Key pdf.use14corefonts has value False
Key pgf.preamble has value
Key pgf.rcfonts has value True
Key pgf.texsystem has value xelatex
Key polaraxes.grid has value True
Key ps.distiller.res has value 6000
Key ps.fonttype has value 3
Key ps.papersize has value letter
Key ps.useafm has value False
Key ps.usedistiller has value None
Key savefig.bbox has value None
Key savefig.directory has value ~
Key savefig.dpi has value figure
Key savefig.edgecolor has value auto
Key savefig.facecolor has value auto
Key savefig.format has value png
Key savefig.orientation has value portrait
Key savefig.pad_inches has value 0.1
Key savefig.transparent has value False
Key scatter.edgecolors has value face
Key scatter.marker has value o
Key svg.fonttype has value path
Key svg.hashsalt has value None
Key svg.id has value None
Key svg.image_inline has value True
Key text.antialiased has value True
Key text.color has value black
Key text.hinting has value force_autohint
Key text.hinting_factor has value 8
Key text.kerning_factor has value 0
Key text.latex.preamble has value
Key text.parse_math has value True
Key text.usetex has value False
Key timezone has value UTC
Key tk.window_focus has value False
Key toolbar has value toolbar2
Key webagg.address has value 127.0.0.1
Key webagg.open_in_browser has value True
Key webagg.port has value 8988
Key webagg.port_retries has value 50
Key xaxis.labellocation has value center
Key xtick.alignment has value center
Key xtick.bottom has value True
Key xtick.color has value black
Key xtick.direction has value out
Key xtick.labelbottom has value True
Key xtick.labelcolor has value inherit
Key xtick.labelsize has value medium
Key xtick.labeltop has value False
Key xtick.major.bottom has value True
Key xtick.major.pad has value 3.5
Key xtick.major.size has value 3.5
Key xtick.major.top has value True
Key xtick.major.width has value 0.8
Key xtick.minor.bottom has value True
Key xtick.minor.ndivs has value auto
Key xtick.minor.pad has value 3.4
Key xtick.minor.size has value 2.0
Key xtick.minor.top has value True
Key xtick.minor.visible has value False
Key xtick.minor.width has value 0.6
Key xtick.top has value False
Key yaxis.labellocation has value center
Key ytick.alignment has value center_baseline
Key ytick.color has value black
Key ytick.direction has value out
Key ytick.labelcolor has value inherit
Key ytick.labelleft has value True
Key ytick.labelright has value False
Key ytick.labelsize has value medium
Key ytick.left has value True
Key ytick.major.left has value True
Key ytick.major.pad has value 3.5
Key ytick.major.right has value True
Key ytick.major.size has value 3.5
Key ytick.major.width has value 0.8
Key ytick.minor.left has value True
Key ytick.minor.ndivs has value auto
Key ytick.minor.pad has value 3.4
Key ytick.minor.right has value True
Key ytick.minor.size has value 2.0
Key ytick.minor.visible has value False
Key ytick.minor.width has value 0.6
Key ytick.right has value False
```

Lots of parameters!

Some we've already seen, but can now adjust globally such as `lines.linewidth`. Others are important for font selection, notably `font.size`, but can also individually adjust `axes.labelsize`, `axes.titlesize`, `legend.fontsize`, etc.

One nice strategy is to gather your preferred options in a dictionary, then use this to update matplotlib's parameters

In [28]:
```
print(mpl.rcParams.get('font.size'))
```

```
10.0
```

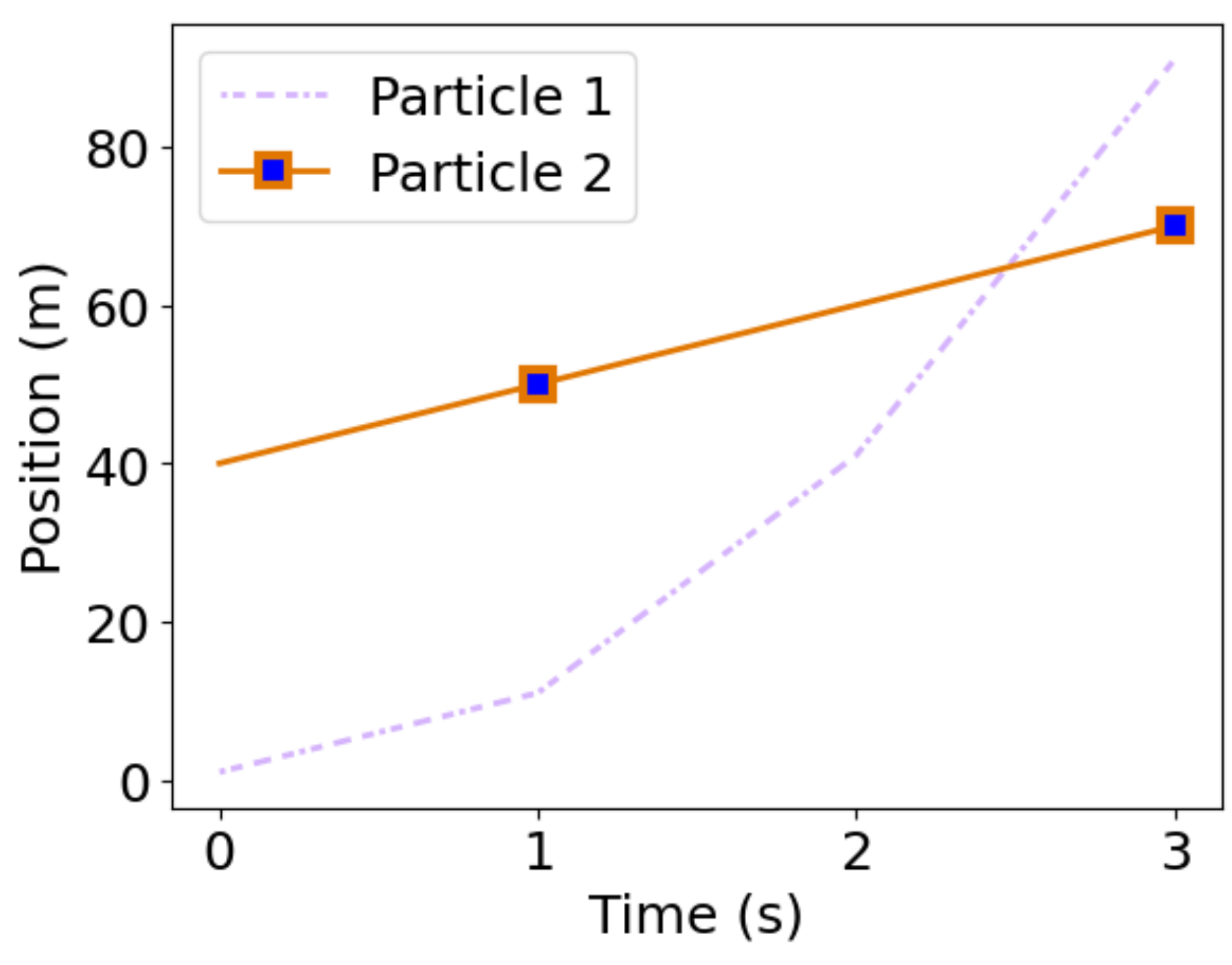
In [29]:
```
my_plot_rc = {'font.size': 18,
              'lines.linewidth': 2,
              'savefig.bbox': 'tight'}
mpl.rcParams.update(my_plot_rc)
```

In [30]:
```
print(mpl.rcParams.get('font.size'))
```

```
18.0
```

Then you can reuse/adapt your own parameter list as you understand and develop your own personal preferences... (or your supervisor's preferences)

In [31]:
```
plt.plot(times, positions, linestyle=(0,(1,1,2,2,3,2,2,1)), color='xkcd:light violet')
plt.plot(times, positions2, marker = 's', markersize = 10, markerfacecolor = 'b', markeredgewidth = 3, markevery = (1,2), color='xkcd:pumpkin')
plt.xlabel("Time (s)")
plt.ylabel("Position (m)")
plt.legend(['Particle 1', 'Particle 2'])
plt.savefig('second.pdf')
```



Note, we can still override individual defaults from rcParams with each plot command

Now we can get nice plots from other packages, like Pandas, that use the matplotlib defaults

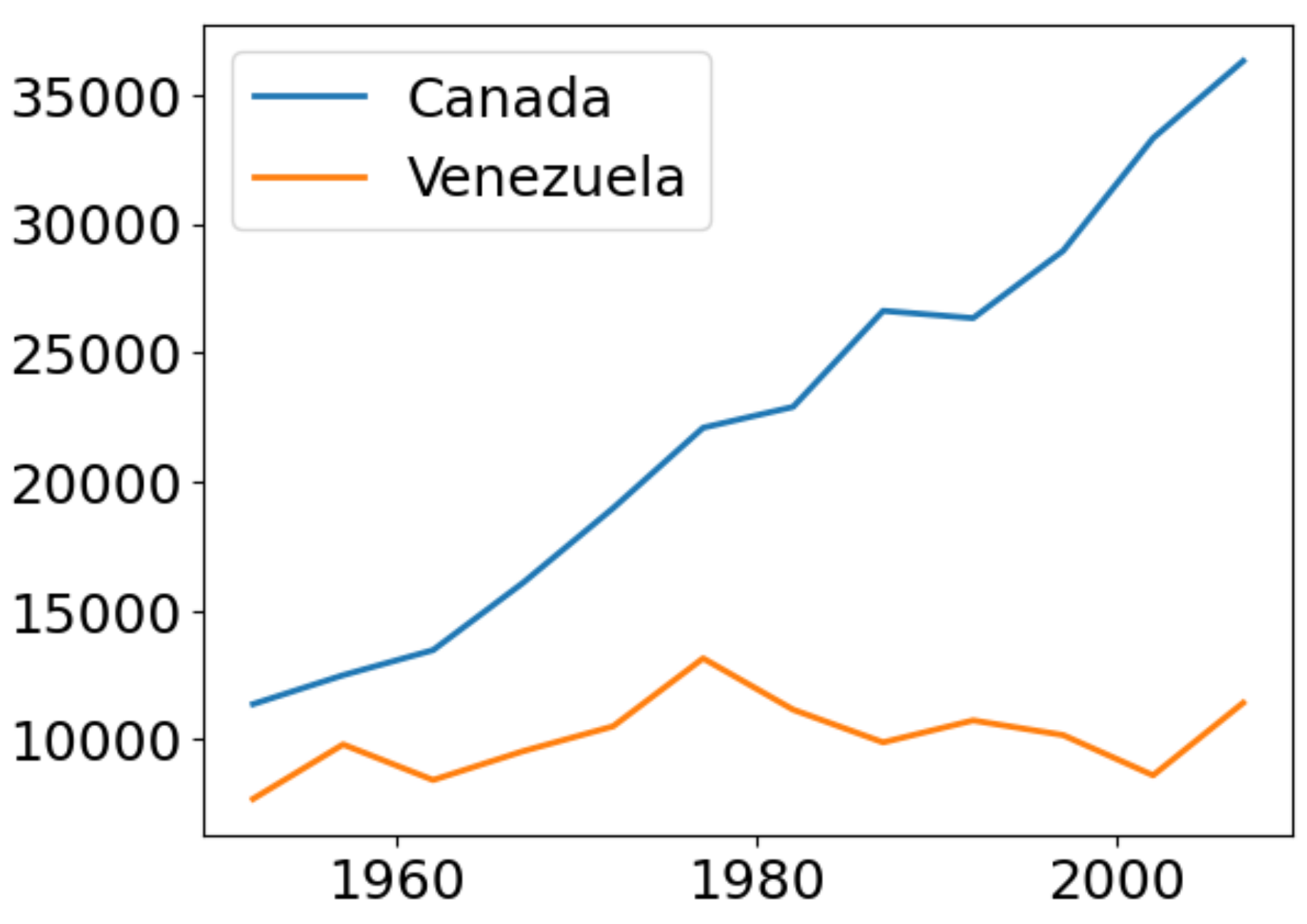
- go back to data_americas, but first rename columns to have numerical headers

In [32]:
```
import pandas as pd
data_americas = pd.read_csv('./data/gapminder_gdp_americas.csv', usecols=range(1,14),index_col="country")
years = data_americas.columns.str.replace('gdpPercap_','')
data_americas.columns = years.astype(int)
```

Now just plot some data:

In [33]:
```
data_americas.loc['Canada'].plot()
data_americas.loc['Venezuela'].plot()
plt.legend()
```

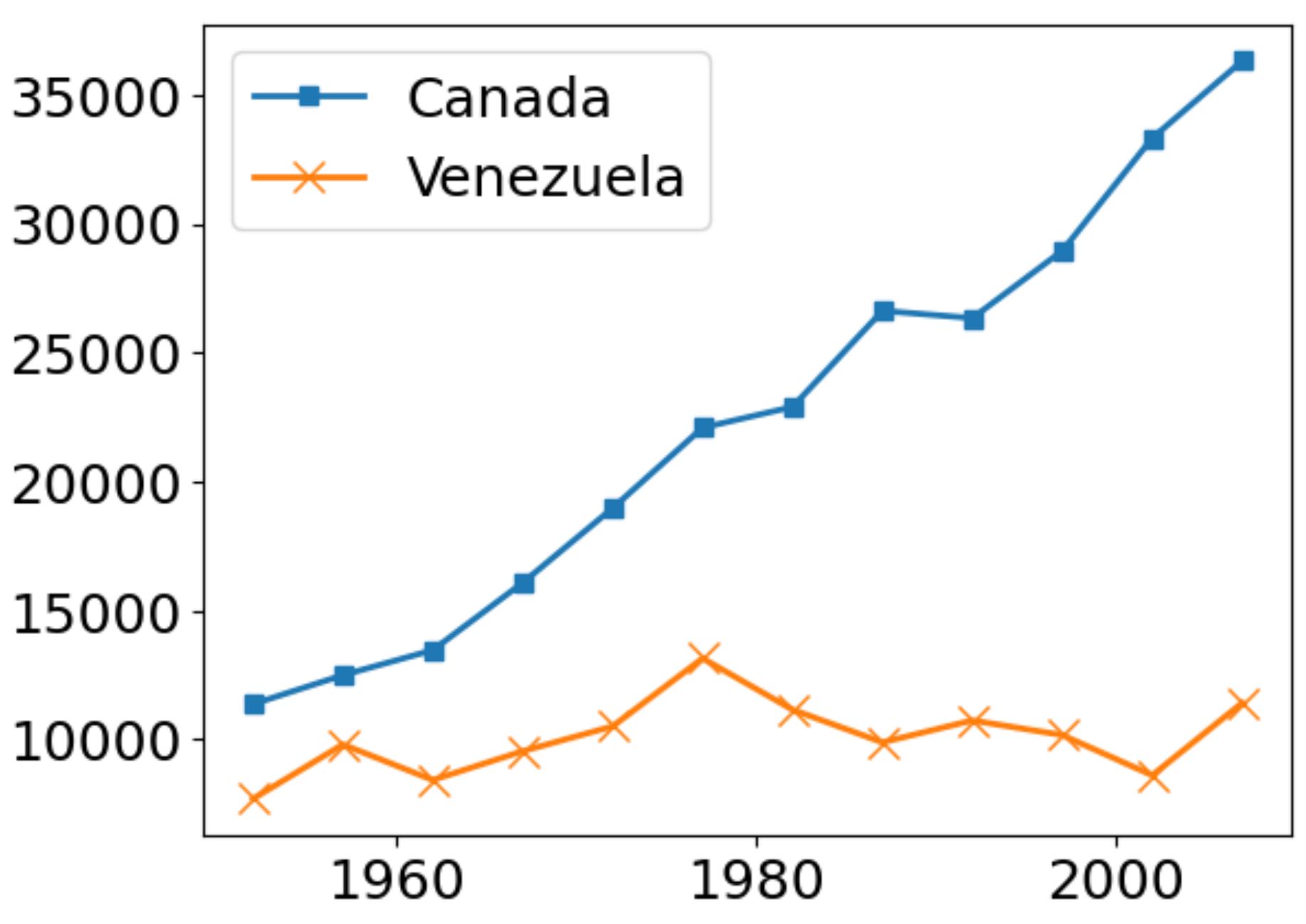
Out[33]:
```
<matplotlib.legend.Legend at 0x1489565d0>
```



Can add markers, and more

In [34]:
```
data_americas.loc['Canada'].plot(marker='s')
data_americas.loc['Venezuela'].plot(marker='x',markersize=10)
plt.legend()
```

Out[34]:
```
<matplotlib.legend.Legend at 0x14898b110>
```



How do we build effectiver plots?

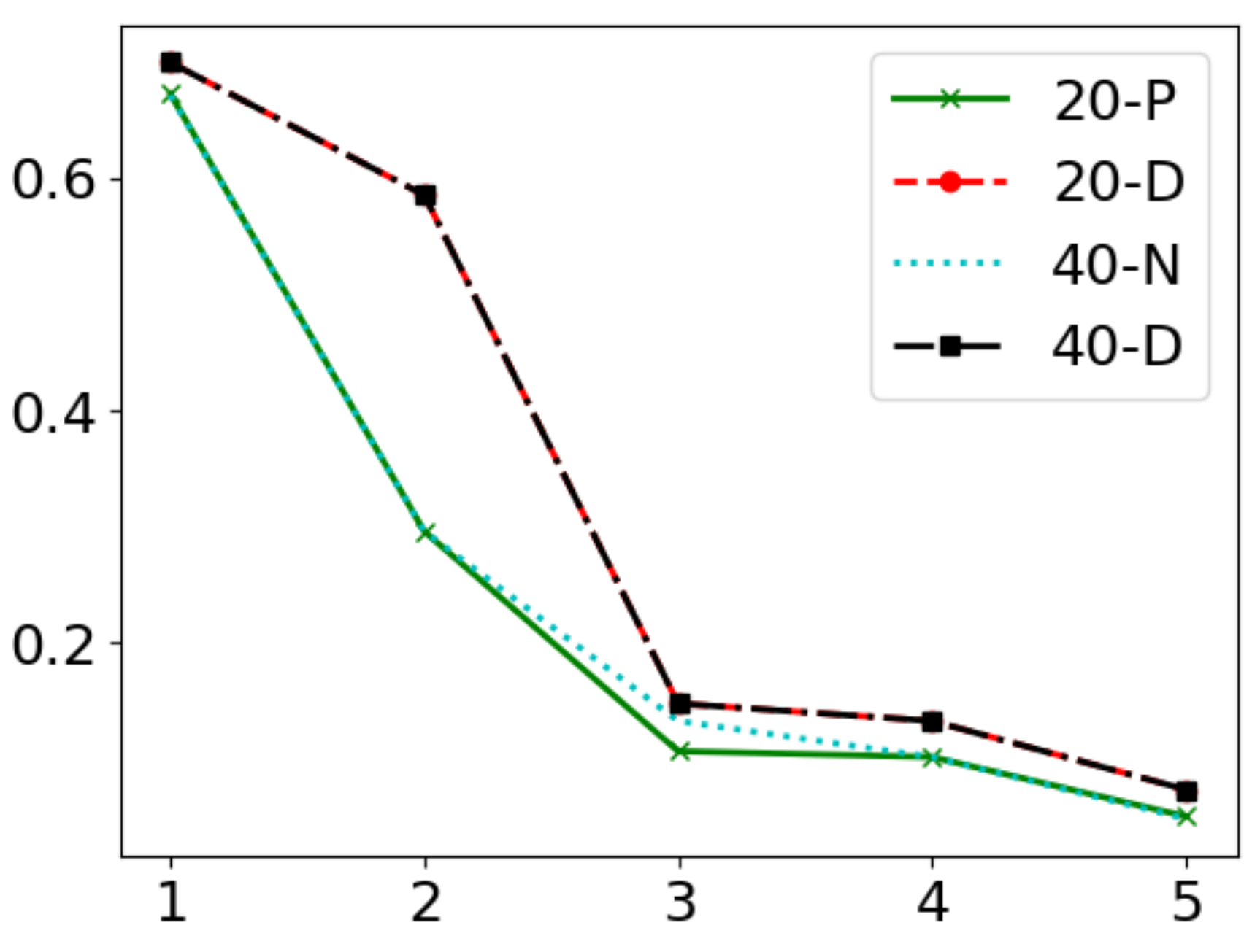
First, consider if a plot is the most effective way to convey the information?

When we want to compare fine-grained differences between data, tables can often be more effective than figures. Consider:

In [35]:
```
k = [1,2,3,4,5]
v1 = [0.672, 0.296, 0.107, 0.102, 0.051]
v2 = [0.699, 0.585, 0.148, 0.133, 0.074]
v3 = [0.671, 0.296, 0.133, 0.102, 0.049]
v4 = [0.699, 0.585, 0.148, 0.133, 0.074]

plt.plot(k, v1, 'x-g', label='20-P')
plt.plot(k, v2, 'o--r', label='20-D')
plt.plot(k, v3, ':c', label='40-N')
plt.plot(k, v4, 's-.k', label='40-D')
plt.legend()
```

Out[35]:
```
<matplotlib.legend.Legend at 0x148b74910>
```



Compare this to:

In [36]:
```
for (kval, val1, val2, val3, val4) in zip(k, v1,v2,v3,v4):
    print(f"{kval} | {val1} | {val2} | {val3} | {val4}")
```

```
1 | 0.672 | 0.699 | 0.671 | 0.699
2 | 0.296 | 0.585 | 0.296 | 0.585
3 | 0.107 | 0.148 | 0.133 | 0.148
4 | 0.102 | 0.133 | 0.102 | 0.133
5 | 0.051 | 0.074 | 0.049 | 0.074
```

Table conveys same information, just as clearly (or better) than graph. Small amounts of it, data is quite similar, so graph doesn't distinguish well between data lines...